

Lesson 1: The Memory API Architecture, Heap Expansion (`brk` / `sbrk`), and `malloc()` Fundamentals

Slide Range: Slide 1 to Slide 5[cite: 2]

The Big Picture: Why Do We Need Dynamic Memory APIs? (Slides 1–3)

In the previous chapter, we learned that every program gets a clean, virtual address space divided into segments: Code, Data, Heap, and Stack[cite: 2].

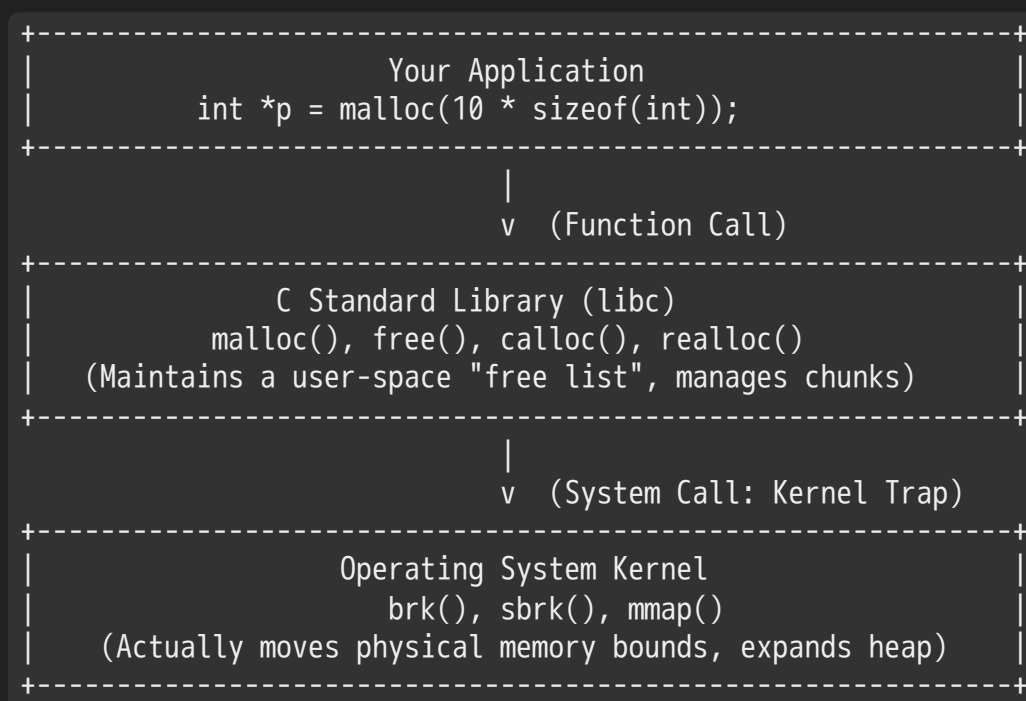
- When your program starts, the OS initializes the **Code** and **Data** segments[cite: 2].
- The **Stack** automatically handles memory for local variables as functions get called and return[cite: 2].

Now, consider this real-world problem: What if you are writing an application that processes user records, and you don't know ahead of time whether the user will input 5 records or 5,000,000 records[cite: 2]?

- You cannot hardcode an array on the stack (it might cause a stack overflow or waste massive amounts of space).
- You cannot declare a fixed-size global array in the data segment.

You need to ask for memory on the fly while the program is actively executing[cite: 2]. This is where the **Memory API** comes in[cite: 2].

The memory API sits on two distinct tiers[cite: 2]:



1. User-Level Library Functions (`malloc` , `free` , `calloc` , `realloc`):

- Declared in `#include <stdlib.h>` [cite: 2].
- Provided by the C runtime library (`libc`) [cite: 2].
- They do **not** run in kernel mode. They are user-level helper functions that manage memory chunks smoothly [cite: 2].

2. System Calls (`brk` , `sbrk` , `mmap` , `munmap`):

- These talk directly to the OS kernel to physically change the size of your process's heap or map pages [cite: 2].

How the Heap Expands: `brk()` and `sbrk()` (Slide 4)

Let's look at how the heap boundary is controlled at the OS level [cite: 2].

Inside the virtual address space, the heap does not start out infinitely large [cite: 2]. It has a boundary line called the **break point** (`break`) [cite: 2]. The `break` marks the current upper limit of the heap [cite: 2]:



When your program uses up all its currently allocated heap space and asks for more, the C library cannot just invent addresses out of nowhere [cite: 2]. It must ask the OS kernel to push the `break` pointer further down into the unmapped free space [cite: 2].

There are two system calls that adjust this pointer [cite: 2]:

1. `brk(void *addr)` (Absolute Positioning)

- You pass an exact **target memory address** [cite: 2].
- The kernel moves the `break` pointer directly to that address [cite: 2].

- *Example from Slide 4 annotations:* If the break point is currently at 16, and you want to expand the heap by 40 bytes, you tell the kernel: `brk(56)` [cite: 2]. The kernel sets the new break point to absolute address 56[cite: 2].

2. `sbrk(intptr_t increment)` (Relative Increment)

- Instead of specifying an exact address, you pass an **increment offset**[cite: 2].
- The kernel adds that offset to the current break point and returns the pointer to the previous break point[cite: 2].
- *Example from Slide 4 annotations:* If the break is currently at 16, calling `sbrk(40)` tells the kernel: "Add 40 bytes to my break point." The break pointer is pushed to 56[cite: 2].

Crucial Engineering Rule:

Programmers should **never** call `brk` or `sbrk` directly in their code[cite: 2]! Why? Because if your code arbitrarily shifts the break pointer around behind `malloc`'s back, you will corrupt the internal data structures that `libc` uses to track free and allocated memory chunks, leading to segmentation faults and undefined behavior[cite: 2].

The Core Memory Workhorse: `malloc()` (Slide 5)

Now let's examine the primary function you use to request heap memory: `malloc()` [cite: 2].

```
c
#include <stdlib.h>
void* malloc(size_t size);
```

1. The Argument: `size_t size`

- You pass the number of bytes you want to allocate[cite: 2].
- `size_t` is an unsigned integer type[cite: 2].
- Notice: `malloc` takes a raw number of **bytes**, not a data type[cite: 2]. It does not know or care whether you plan to store integers, floating-point numbers, characters, or custom structs[cite: 2].

2. The Return Value: `void*`

- **On Success:** `malloc` returns a `void*` (a raw, untyped generic pointer) pointing to the first byte of the allocated memory block on the heap[cite: 2]. In C, a `void*` pointer can be implicitly or explicitly cast to any type (e.g., `int*`, `char*`) [cite: 2].
- **On Failure:** If the system is out of memory (e.g., your 32-bit program asks for a 20 GB block, or physical RAM and swap are exhausted), `malloc()` fails and returns `NULL` (address `0x0000 0000`) [cite: 2].

Why is Accessing `NULL` Fatal? (Handwritten Annotation on Slide 5)

Why does your program crash with a "Segmentation Fault" the moment you try to dereference a `NULL` pointer[cite: 2]?

- `NULL` points to address `0x0000 0000` [cite: 2].
- In the virtual address space, the page containing address `0x0` is intentionally left unmapped and marked with zero access permissions[cite: 2].
- The moment your CPU tries to read or write byte `0x0`, the MMU (Memory Management Unit) hardware immediately raises an illegal memory access exception, causing the OS to terminate your process[cite: 2]!

The Free List Concept (Slide 4 & 5 Annotations)

Does `malloc()` make a kernel system call (`sbrk()`) every single time you allocate 10 bytes[cite: 2]?

No. Jumping into the kernel is expensive[cite: 2].

- When you call `malloc()`, the library requests a large block from the OS at once (say, 4 KB or 64 KB) using `sbrk()` [cite: 2].
- Inside user space, `libc` maintains a linked list called a **Free List**[cite: 2].
- When you request small memory allocations, `malloc` carves out pieces from its free list entirely within user space, avoiding unnecessary system call overhead[cite: 2].
- Only when that free list runs out of space does `libc` trigger an `sbrk()` system call to expand the heap further[cite: 2].

Let me know if this breakdown is clear, and we'll move on to **Lesson 2 (Slides 6 to 9)** to see how `sizeof()` interacts with memory, unpack the hidden metadata header inside `malloc` allocations, and examine how `free()` and dangling pointers work[cite: 2]!

2026-09-03 13:05:10

You 

next

 Gemini 3.8 Flash

Lesson 2: The `sizeof()` Trap, Chunk Metadata Headers, and the Mechanics of `free()`

Slide Range: Slide 6 to Slide 9[cite: 2]

The `sizeof()` Trap: Compile-Time vs. Run-Time (Slide 6)

When allocating memory dynamically, you rarely hardcode literal byte numbers like `malloc(40)` [cite: 2]. Instead, you use standard types and the `sizeof()` operator[cite: 2]:

```
c
int *x = malloc(10 * sizeof(int));
```

Now, look at these two snippets and predict what `printf()` will print in each case[cite: 2]:

Snippet A: The Dynamic Pointer Trap

```
c
int *x = malloc(10 * sizeof(int));
printf("%lu\n", sizeof(x));
```

- **Output:** It prints `4` (on a 32-bit machine) or `8` (on a 64-bit machine)[cite: 2]! It does **not** print `40` [cite: 2].
- **Why?** `sizeof()` is a **compile-time operator**, not a run-time function call[cite: 2]. During compilation, the compiler inspects the *type* of the identifier `x` [cite: 2]. Because `x` is declared as an integer pointer (`int *`), the compiler replaces `sizeof(x)` with the fixed byte size of a memory address pointer[cite: 2]. The compiler has no knowledge of what `malloc()` will return at runtime[cite: 2].

Snippet B: The Static Array

```
c
int x[10];
printf("%lu\n", sizeof(x));
```

- **Output:** It prints `40` (assuming 10×4 bytes)[cite: 2].
- **Why?** Here, `x` is declared as a fixed array[cite: 2]. The compiler already knows the total element count and type at compile time, so it substitutes the constant `40` directly into the binary[cite: 2].

Rule: Never pass a dynamically allocated pointer to `sizeof()` expecting it to tell you how many bytes you requested from `malloc()` [cite: 2]. It will only report the size of the pointer variable itself[cite: 2].

The Hidden Secret: The Metadata Chunk Header (Slide 6 & Slide 7)

If `sizeof(x)` cannot tell how many bytes were allocated, how does `free(x)` work[cite: 2]?

Notice the signature of `free()` [cite: 2]:

```
c

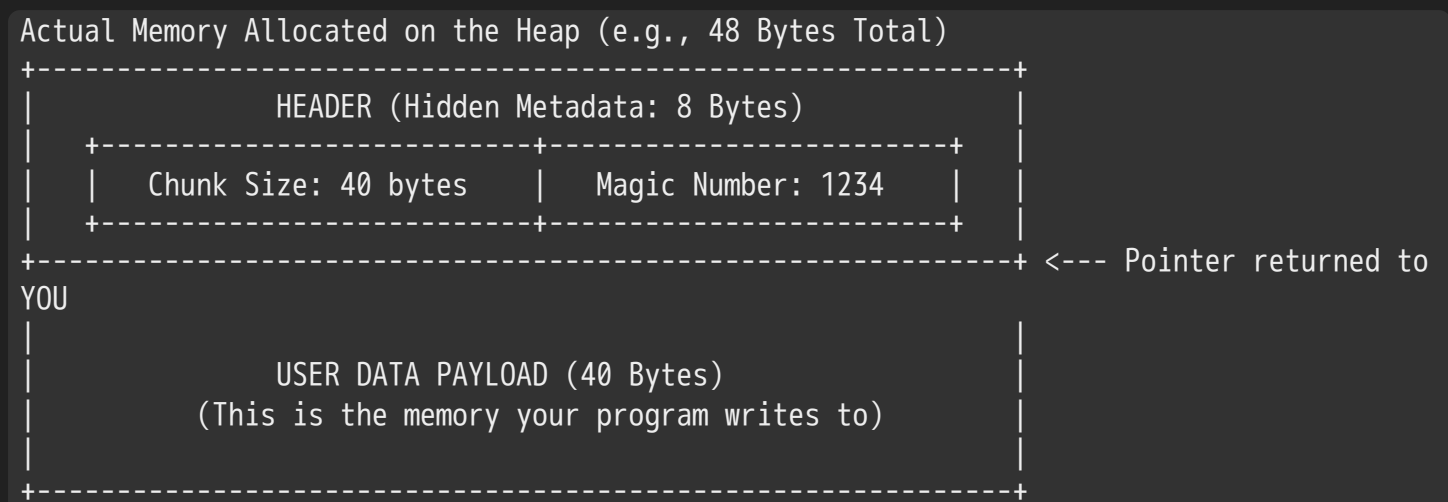
void free(void *ptr);
```

Look closely: you only hand `free()` an address (`ptr`) [cite: 2]. You do **not** pass a byte count like `free(ptr, 40)` [cite: 2].

How does the C library know whether to free 4 bytes, 40 bytes, or 40,000 bytes [cite: 2]?

The Hidden Header Structure

When you call `malloc(40)` , the memory allocator does not allocate just 40 bytes [cite: 2]. It quietly allocates extra space directly preceding your data to store an internal **header block** (bookkeeping metadata) [cite: 2]:



- **Chunk Size:** Stores the exact size of the payload so `free()` knows how many bytes to reclaim [cite: 2].
- **Magic Number (Integrity Canary):** A known sentinel value (like `0x1234` or an integrity bit pattern) used by the allocator to verify that the pointer was actually generated by `malloc()` and that the header has not been corrupted by an out-of-bounds write [cite: 2].

When `malloc(40)` completes:

1. It carves out 48 bytes on the heap (8 bytes for the header + 40 bytes for your data) [cite: 2].
2. It fills in the header: `size = 40` , `magic = 1234` [cite: 2].
3. It returns the memory address starting **right after the header** [cite: 2].

When you call `free(ptr)` :

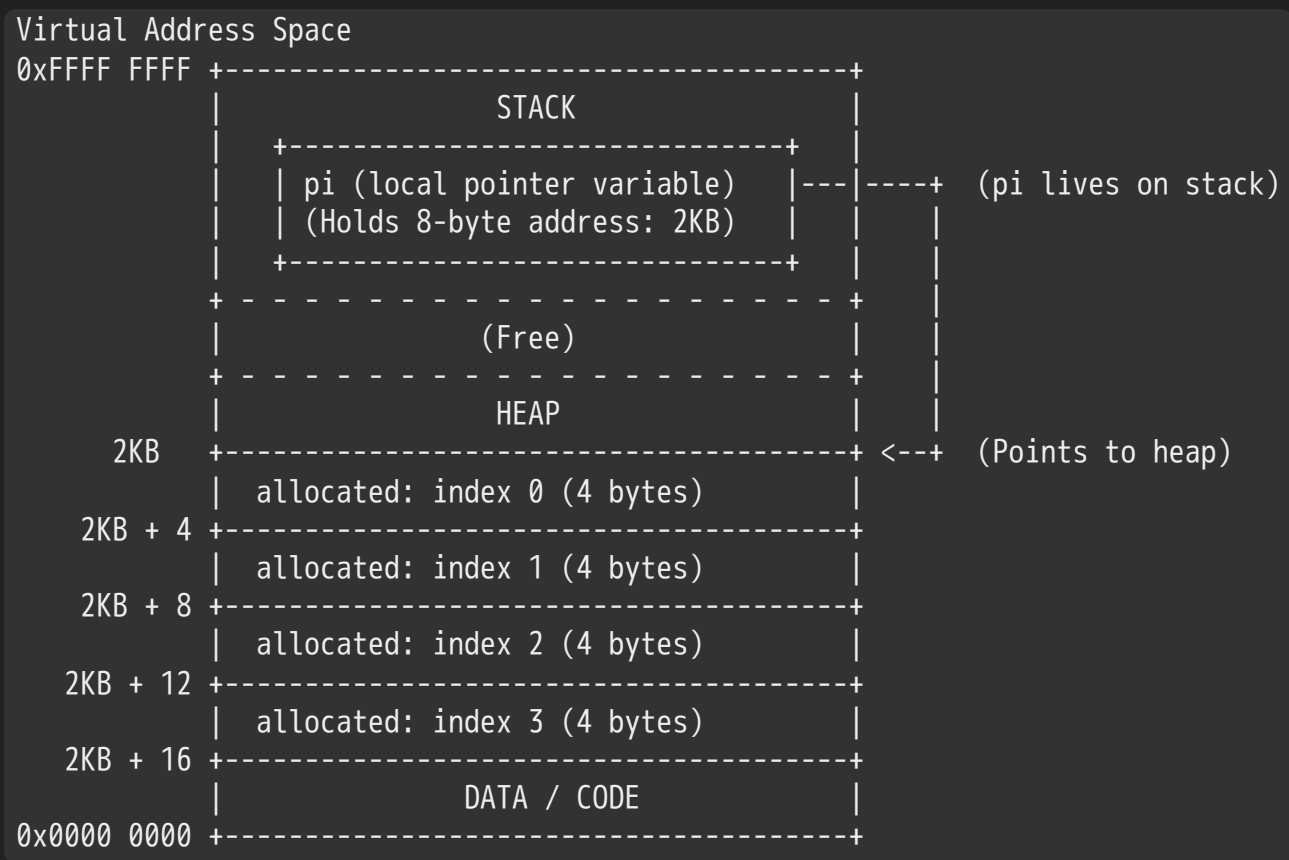
1. The allocator takes `ptr` and backs up by the size of the header (e.g., subtracting 8 bytes in pointer arithmetic) [cite: 2].

2. It inspects the header, verifies the magic number, reads the size field (40 bytes), and puts all 48 bytes back onto the user-space **free list** for future allocations[cite: 2].

Allocating Memory Step-by-Step (Slide 8)

Let's trace what happens in the virtual address space during allocation[cite: 2]:

```
c
int *pi;
pi = (int *)malloc(sizeof(int) * 4); // 4 integers = 16 bytes
```



1. The Pointer **pi** Lives on the Stack:

- Declaring `int *pi;` creates a local pointer variable inside the current function frame on the **Stack**[cite: 2].
- On a 64-bit CPU architecture, any pointer variable occupies 8 bytes of stack storage[cite: 2].

2. The Data Payload Lives on the Heap:

- `malloc(16)` allocates a 16-byte contiguous chunk inside the **Heap** starting at address **2KB** [cite: 2].
- **pi** receives that starting address (**2KB**), pointing from the stack directly down into the heap[cite: 2].

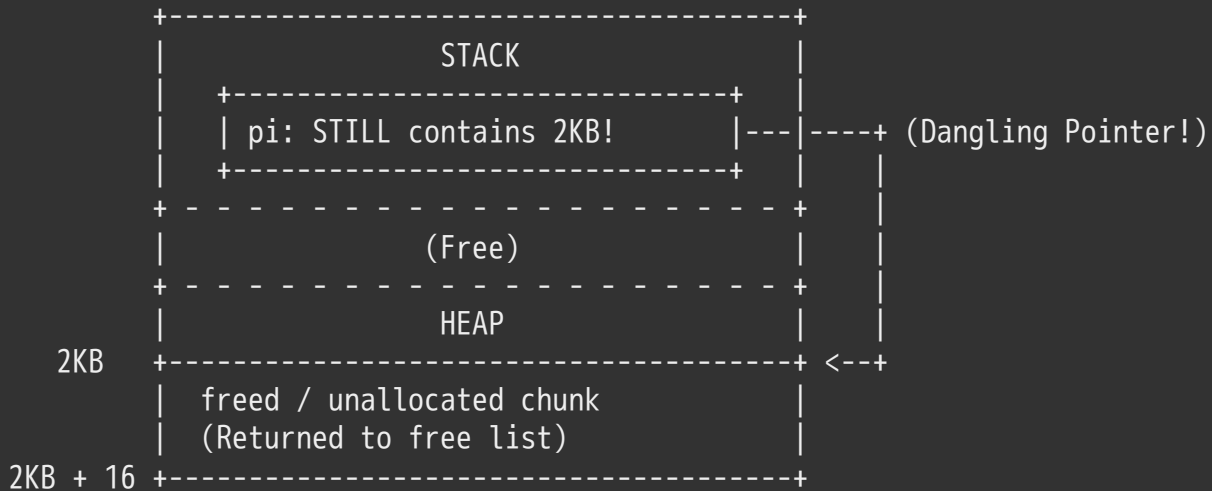
The Dangling Pointer Problem (Slide 9)

What happens when you are done using the memory and call `free(pi)` [cite: 2]?

c

```
free(pi);
```

Address Space After `free(pi)`



Look at the pointer `pi` after `free(pi)` executes [cite: 2]:

- Calling `free(pi)` tells the C library: "I am done with the heap memory starting at address `2KB` ." The allocator updates its internal free list [cite: 2].
- **Crucial Point:** `free()` does **not** erase or change the variable `pi` itself [cite: 2]!
- `pi` still lives on the stack, and it still holds the exact memory address `2KB` [cite: 2].
- But the memory at address `2KB` is now invalid or could be reassigned to something else at any moment [cite: 2].

This is a **Dangling Pointer** [cite: 2]: a pointer variable that still holds a memory address, but the memory it points to has already been released [cite: 2].

If you accidentally write:

c

```
*pi = 50; // Use-After-Free bug!
```

You are writing data into freed memory. If that block was re-assigned to another part of your program, you will quietly overwrite and corrupt other variables without an immediate crash, leading to silent data corruption and severe security vulnerabilities [cite: 2].

The Professional Best Practice:

Whenever you free a pointer that will remain in scope, set it to `NULL` immediately afterward [cite: 2]:

C

```
free(pi);  
pi = NULL; // Clear the dangling pointer
```

If you accidentally try to write through `pi` later, writing to `NULL` triggers an immediate hardware exception and program crash, alerting you to the bug instantly instead of corrupting data silently[cite: 2].

Are you ready to move on to **Lesson 3 (Slides 10 to 14)** to explore the classic memory bugs—forgetting to allocate, forgetting the null-terminator, and memory leaks[cite: 2]?

2026-09-03 13:06:04

You 

next



Gemini 3.8 Flash

Lesson 3: Common Memory Errors – Unallocated Pointers, the Null-Terminator Bug, Uninitialized Reads, and Memory Leaks

Slide Range: Slide 10 to Slide 14[cite: 2]

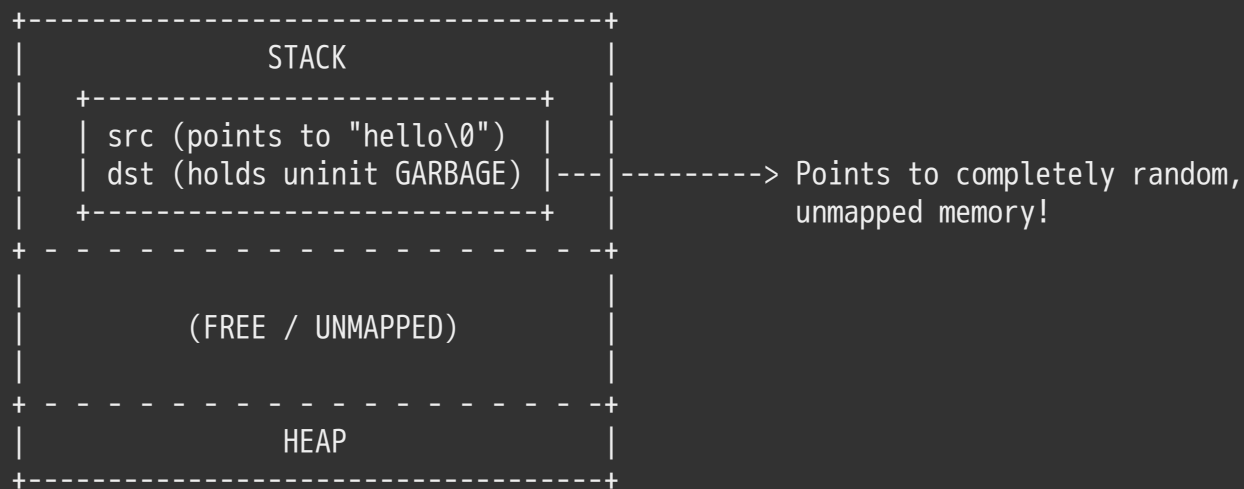
Error 1: Forgetting to Allocate Memory (Slide 10)

Let's look at one of the most common beginner bugs in C[cite: 2]:

C

```
char *src = "hello"; // Character string constant  
char *dst;           // Unallocated pointer variable!  
strcpy(dst, src);    // Segmentation fault and crash!
```

Virtual Address Space



- **What Went Wrong?**

Declaring `char *dst;` simply reserves a local pointer on the **Stack**[cite: 2]. It does **not** allocate memory for characters to live in[cite: 2]!

- **The Garbage Pointer Value:**

Because `dst` is an uninitialized local variable, it contains whatever leftover bit pattern was on the stack[cite: 2]. It points to a completely random memory address[cite: 2].

- **Why You Almost Always Get a Segmentation Fault (The Math Annotation on Slide 10):**

In a 32-bit or 64-bit address space, only a small portion of memory is actually mapped to physical RAM (Code, Data, a small Heap, and a small Stack)[cite: 2]. The vast gap in the middle is completely unmapped and marked illegal by the OS[cite: 2].

The probability that an uninitialized pointer lands in unmapped territory is virtually ~ 1 [cite: 2]. When `strcpy()` tries to write 'h', 'e', 'l', 'l', 'o' to that random address, the CPU hardware catches the illegal access and terminates the program with a **Segmentation Fault**[cite: 2].

The Fix (Slide 11):

```
c
char *src = "hello";
char *dst = (char *)malloc(strlen(src) + 1); // Explicitly allocate memory!
strcpy(dst, src);                          // Works properly!
```

By allocating space with `malloc()`, `dst` points to a valid chunk on the heap where bytes can safely be written[cite: 2].

Error 2: Not Allocating Enough Memory (The Off-by-One Bug) (Slide 12)

Look closely at this subtle error[cite: 2]:

```
c
```

```
char *src = "hello";
char *dst = (char *)malloc(strlen(src)); // BUG: Forgot the + 1!
strcpy(dst, src);                        // Appears to work, but corrupted
memory!
```

What is allocated on Heap (5 bytes):

```
+---+---+---+---+---+
| h | e | l | l | o |
+---+---+---+---+---+
```

What strcpy writes: \
+---> [\0] OVERFLOWS into next chunk!

- **The String Representation in C:**

The string "hello" looks like 5 letters, but in C, every string must end with a **null-terminator character** ('\0')[cite: 2]. Therefore, "hello" requires **6 bytes** of storage ('h' , 'e' , 'l' , 'l' , 'o' , '\0')[cite: 2].

- **The Trap:**

The standard function `strlen("hello")` returns `5` (it counts characters up to, but not including, the null terminator)[cite: 2].

- If you write `malloc(strlen(src))` , you only allocate **5 bytes**[cite: 2].
- When `strcpy()` executes, it writes all 5 letters and then appends '\0' into the 6th byte[cite: 2]. That 6th byte is outside your allocated memory chunk[cite: 2]!

What Happens at Runtime? (Slide 12 Annotation)

- This bug often runs without crashing right away[cite: 2]!
- If the extra byte happens to write into the free space before the `break` pointer, the hardware will not trigger a segmentation fault[cite: 2].
- **The Danger:** That extra byte overwrites whatever comes directly after your chunk—such as another variable, or the **metadata chunk header** of the next block[cite: 2]! Later, when you call `free()` on another pointer, the allocator encounters corrupted metadata and crashes unpredictably[cite: 2].

Error 3: Forgetting to Initialize (Slide 13)

C

```
int *x = (int *)malloc(sizeof(int)); // Memory is allocated, but NOT
initialized!
printf("*x = %d\n", *x);              // Reads uninitialized memory!
```

- **What Happens?**

`malloc()` simply claims memory from the heap; it does **not** zero out or clean the memory block[cite: 2].

- If that physical RAM previously held data from an old variable or an earlier function call, that old bit pattern remains there[cite: 2].
- Reading `*x` yields an unpredictable **garbage value**[cite: 2].
- **Rule:** After allocating with `malloc()`, always initialize the memory before reading from it (or use `calloc()`, which zero-initializes memory automatically)[cite: 2].

Error 4: Memory Leaks (Slide 14)

What happens when you continuously request memory and never release it[cite: 2]?

```
c
void leak_memory() {
    int *x = malloc(100); // 100 bytes allocated on heap
    // free(x);           // Forgot to free!
}
```

Iterative Allocations on the Heap:

```
+-----+
| allocated (active) |
+-----+
| unused (lost reference - leaked!) |
+-----+
| unused (lost reference - leaked!) |
+-----+
| unused (lost reference - leaked!) |
+-----+
| Heap expands downward via sbrk()... |
v                                     v
[ OS eventually kills the process (OOM) ]
```

- When `leak_memory()` finishes, its stack frame is popped, destroying the local pointer variable `x` [cite: 2].
- However, the 100 bytes on the heap remain marked as allocated[cite: 2]!
- Because the pointer `x` is gone, the program has no way to reference or free that memory ever again[cite: 2]. This is a **Memory Leak**[cite: 2].

Consequences of Memory Leaks:

- If a short-lived command-line tool leaks memory and exits, the OS automatically reclaims all physical pages when the process terminates[cite: 2].

- However, for long-running daemon services (like web servers, database engines, or the OS itself), accumulating memory leaks will cause the heap to expand continuously[cite: 2].
- Eventually, the machine exhausts physical RAM and swap space[cite: 2]. The OS kernel's **OOM (Out of Memory) Killer** steps in and terminates the process[cite: 2].

Garbage Collection vs. Manual Management (Slide 14 Annotation)

- **High-Level Languages (Java, Python, Go):** These runtime environments track how many active references point to each heap object using reference counters or tracing garbage collectors[cite: 2]. When an object's reference count drops to zero, the garbage collector automatically reclaims the memory[cite: 2].
 - *Trade-off:* The garbage collector runs periodic scans, introducing runtime overhead and latency pauses[cite: 2].
- **Low-Level Languages (C, C++):** The programmer has full control and must manually call `free()` to release heap blocks when they are no longer needed[cite: 2].

Let me know if you are ready for **Lesson 4 (Slides 15 to 22)** to cover dangling pointers, double-free bugs, `calloc` / `realloc` , and memory mapping (`mmap` / `munmap`) [cite: 2]!

2026-09-03 13:06:50

You 

next

 Gemini 3.8 Flash

Lesson 4: Dangerous Freeing Bugs, `calloc` / `realloc` , and Direct Memory Mapping (`mmap`)

Slide Range: Slide 15 to Slide 22[cite: 2]

Dangling Pointers in Action: Breaking the Linked Structure (Slide 15)

In Lesson 2, we introduced dangling pointers[cite: 2]. Slide 15 illustrates how a dangling pointer corrupts linked data structures on the heap[cite: 2]:

Heap Structure Before free(b):

+-----+		
Node A (at 2KB)	Points to Node B (at 3KB)	
+-----+		
Node B (at 3KB)	Points to Node C (at 4KB)	
+-----+		
Node C (at 4KB)	Points to NULL (End of list)	
+-----+		

Now, what happens if the programmer writes `free(b)` without updating Node A's link[cite: 2]?

Heap Structure After free(b):

+-----+		
Node A (at 2KB)	STILL Points to 3KB! (Dangling Link!)	
+-----+		
3KB Chunk	FREED / UNALLOCATED	
+-----+		
Node C (at 4KB)	UNREACHABLE (Or reachable via invalid)	
+-----+		

- Node B (at 3KB) is released back to the free list[cite: 2].
- Node A still holds 3KB in its next pointer[cite: 2].
- If your program later traverses the list (`current = current->next`), it steps directly into freed memory[cite: 2].
- If that 3KB chunk was subsequently reallocated to store arbitrary text or numbers, dereferencing `current->next` will read garbage values or invalid memory addresses, throwing an unpredictable crash or corrupting data silently[cite: 2].

Incorrect Free Operations: Double Free & Bad Pointers (Slide 16)

Slide 16 demonstrates two mistakes programmers make with `free()` [cite: 2]:

1. Double Free Bug

C

```
int *x = (int *)malloc(sizeof(int)); // Allocated at 2KB
free(x);                             // Memory freed once
free(x);                             // Freeing again! Undefined behavior!
```

- **Why is this dangerous?**[cite: 2]

The first `free(x)` inserts the chunk back into `libc`'s internal free list[cite: 2]. Calling `free(x)` a second time attempts to insert the exact same chunk into the free list again[cite: 2]. This creates a **cycle/loop** inside the allocator's linked list[cite: 2]. Future calls to `malloc()` will hand out the exact same chunk of memory to two different requests, leading to memory corruption and potential security exploits[cite: 2].

- As stated on the slide: **"We don't know what will happen. Behavior is undefined."**[cite: 2]

2. Freeing Memory Not Allocated by `malloc()`

```
c
int *x = (int *)malloc(sizeof(int));
free(x + 12); // ILLEGAL: Pointing inside or outside the chunk!
```

- Remember from Lesson 2: When you pass a pointer to `free(ptr)`, the allocator automatically jumps backward (e.g., 8 or 16 bytes) to inspect the **chunk header** (size and magic number)[cite: 2].
- If you pass an offset like `x + 12`, the allocator steps back into whatever random bits happen to sit 12 bytes away[cite: 2].
- If those bits don't match the magic number, the allocator aborts with an error[cite: 2].
- **The "Unlucky Case" (Annotation on Slide 16):** If the memory bits 12 bytes forward coincidentally look like a valid magic number (e.g., `0x1234`), `free()` assumes it found a valid header, reads whatever garbage byte value happens to be there as the size, and corrupts its internal free list[cite: 2]!

Complementary APIs: `calloc()` and `realloc()` (Slide 17)

Beyond `malloc()` and `free()`, the standard C library provides two dynamic memory utilities[cite: 2]:

1. `calloc()`

```
c
#include <stdlib.h>
void *calloc(size_t num, size_t size);
```

- **Parameters:** `num` (the number of elements) and `size` (the size of each element in bytes)[cite: 2].
- **Difference from `malloc()`:** `malloc()` leaves newly allocated memory uninitialized (filled with old garbage data)[cite: 2]. `calloc()` allocates the memory **and automatically clears every byte to zero** before returning the pointer[cite: 2].
- Useful for arrays where you need elements initialized to zero.

2. `realloc()`

```
c
```

```
#include <stdlib.h>
void *realloc(void *ptr, size_t size);
```

- **Parameters:** `ptr` (a pointer previously returned by `malloc`, `calloc`, or `realloc`) and `size` (the new desired byte size)[cite: 2].
- **What it does:** Expands or contracts the existing block of memory[cite: 2].
- **Mechanics:**
 - If there is enough contiguous free space immediately following the current block on the heap, `realloc()` extends the chunk in place and returns the same pointer.
 - If there is not enough room directly adjacent, `realloc()` allocates a new block of the requested size elsewhere on the heap, copies the old data into the new block, automatically frees the old chunk, and returns the pointer to the new memory location.

System Calls Behind the Scenes: Revisiting `brk()` and `sbrk()` (Slide 18)

Slide 18 revisits how `malloc` interfaces with the kernel[cite: 2]:

C

```
#include <unistd.h>
int brk(void *addr);
void *sbrk(intptr_t increment);
```

- The **break** pointer marks the current boundary between the mapped heap and the unmapped virtual address space[cite: 2].
- **Does every single call to `malloc()` invoke `brk()` ?**
No[cite: 2]. When you ask for memory, `malloc()` first looks at its user-space free list[cite: 2]. If an existing free chunk accommodates the request, it returns that chunk directly without issuing a system call[cite: 2].
- Only when the free list cannot satisfy the request does `malloc()` call `sbrk()` to ask the OS kernel to expand the **break** pointer[cite: 2].

Direct Memory Mapping: The `mmap()` System Call (Slides 19–21)

While `brk()` and `sbrk()` grow the heap linearly, modern operating systems provide an alternative mechanism for allocating memory: `mmap()` (**Memory Map**)[cite: 2].

Why Do We Need `mmap()` ? (Slide 19 Annotations)

Consider two processes: Process 1 (Producer) and Process 2 (Consumer)[cite: 2].

- Both processes have completely separate, isolated address spaces[cite: 2].

- Suppose Process 1 writes data to a disk file, and Process 2 reads it[cite: 2]. Disk operations take milliseconds (~4 ms), whereas CPU memory accesses take nanoseconds (~1 ns)—making memory a million times faster[cite: 2]!
- If a program writes a million records via `write()` in a loop, it makes 1,000,000 system calls, switching between User and Kernel modes 1,000,000 times[cite: 2].
- **The Solution:** Use `mmap()` to map a file or shared memory directly into the process's virtual address space[cite: 2]. Once mapped, reading and writing to that memory accesses RAM directly without continuous system calls[cite: 2]!

The `mmap()` Function Signature

C

```
#include <sys/mman.h>
void *mmap(void *ptr, size_t length, int prot, int flags, int fd, off_t offset);
```

- **ptr** : Suggested starting address for the mapping (usually passed as `NULL` or `0` so the OS chooses the address)[cite: 2].
- **length** : Number of bytes to map[cite: 2].
- **prot** : Memory protection flags: `PROT_READ` (readable), `PROT_WRITE` (writable), `PROT_EXEC` (executable)[cite: 2].
- **flags** : Behavior settings:
 - `MAP_SHARED` : Updates to this memory region are visible to other processes sharing the mapping[cite: 2].
 - `MAP_PRIVATE` : Copy-on-write mapping; modifications are private to this process.
 - `MAP_ANONYMOUS` : Not backed by any file (pure RAM allocation)[cite: 2].
- **fd** : The file descriptor to map (or `-1` if allocating anonymous memory)[cite: 2].
- **offset** : Byte offset in the file to start mapping from[cite: 2].

The Two Modes of `mmap()` (Slides 20 & 21)

1. File-Backed Mapping (Slide 20)

C

```
int fd = open("data.txt", O_RDWR);
ptr = mmap(NULL, 40, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
if (ptr == MAP_FAILED) {
    exit(EXIT_FAILURE);
}
```

```
}
```

- **What Happens:** The file `data.txt` is mapped directly into an empty region in your virtual address space between the Heap and the Stack[cite: 2].
- You treat `ptr` as a normal C pointer (e.g., `ptr[0] = 'A'`)[cite: 2].
- When you modify `ptr`, the OS handles synchronizing those changes back to the actual file on disk automatically[cite: 2].

2. Anonymous Mapping (Slide 21)

```
c
```

```
ptr = mmap(NULL, 40, PROT_READ | PROT_WRITE,  
           MAP_SHARED | MAP_ANONYMOUS, -1, 0);  
if (ptr == MAP_FAILED) {  
    exit(EXIT_FAILURE);  
}
```

- **What Happens:** Notice `fd = -1` and the flag `MAP_ANONYMOUS` [cite: 2].
- This tells the kernel: *"I do not want to map a file on disk. I just want raw, zero-filled physical RAM pages allocated directly into my virtual address space."*[cite: 2]
- **How `malloc()` Uses This:**
For small allocations (e.g., 32 bytes), `malloc()` uses `sbrk()` to expand the heap[cite: 2]. But for very large allocations (e.g., requesting a 100 MB buffer), `malloc()` switches to calling `mmap()` with `MAP_ANONYMOUS` [cite: 2]. When you later call `free()` on that large chunk, `munmap()` releases the physical pages directly back to the OS immediately[cite: 2].

File Descriptors Explained (Annotation on Slide 21)

Slide 21 notes standard file descriptor assignments:

- `0` : `stdin` (Standard Input)[cite: 2]
- `1` : `stdout` (Standard Output)[cite: 2]
- `2` : `stderr` (Standard Error)[cite: 2]
- When a process opens user files (`a.txt` , `b.txt`), the kernel assigns ascending integer indices (`3` , `4` , etc.) into the process's open-file table[cite: 2]. If file descriptor `4` (`b.txt`) is closed and a new file (`d.txt`) is opened, the kernel reuses the lowest available descriptor number (`4`)[cite: 2].

Chapter 14 Summary (Slide 22)

The dynamic memory stack spans from user-level conveniences down to kernel-level primitives[cite: 2]:

